

目录

1	实验概述	2
2	Project1 虚拟内存的完善	2
2.1	划分磁盘交换区	2
2.2	磁盘 IO 操作	2
2.3	交换区的管理	5
2.4	开启缺页中断	6
2.5	虚拟页的分配	6
2.6	请求页表的处理与页面的调出	7
2.7	虚拟页的释放	7
2.8	缺页中断处理	8
2.9	测试样例的设计和运行结果	10
3	Project1 到 Project2 的过渡: 从内核态到用户态	11
4	Project2:malloc 和 free	14
4.1	用户态的内存分配	14
4.2	空闲块的管理	15
4.3	malloc 的实现	16
4.4	free 的实现	17
4.5	malloc & free 的简单样例测试	19
5	从 Project2 到 Project5	22
5.1	Project2 存在的问题	22
5.2	解决方案	22
6	Project5: 写时复制 (COW)	24
6.1	没有写时复制的情况	24
6.2	写时复制 (COW) 应考虑的几个问题	25
6.3	父子进程应共享什么资源	25
6.4	写时复制机制的实现	28
6.5	物理页引用次数的维护	29
6.6	COW 机制下的页面中断处理	32
6.7	测试样例	35
6.7.1	测试样例一	35
6.7.2	测试样例二	36
7	实验总结与心得体会	37

1 实验概述

2 Project1 虚拟内存的完善

2.1 划分磁盘交换区

首先查看 run/hd.img 的磁盘信息与总扇区数

```
(base) david@David:~/i386/Lab9/project1/run$ ls -la hd.img
-rw-r--r-- 1 david david 10321920  5月 27 19:53 hd.img
(base) david@David:~/i386/Lab9/project1/run$ echo "磁盘大小: $((stat -c%s hd.img) / 512)) 扇区"
磁盘大小: 20160 扇区
```

查看 makefile 中的指令:

```
build : mbr.bin bootloader.bin kernel.bin kernel.o
        dd if=mbr.bin of=$(RUNDIR)/hd.img bs=512 count=1 seek=0 conv=notrunc
        dd if=bootloader.bin of=$(RUNDIR)/hd.img bs=512 count=5 seek=1 conv=notrunc
        dd if=kernel.bin of=$(RUNDIR)/hd.img bs=512 count=145 seek=6 conv=notrunc
```

从此可知磁盘每个扇区占 512 字节，且磁盘的空间规划如下

组件	扇区数量	起始扇区	结束扇区	说明
MBR	1	0	0	主引导记录
Bootloader	5	1	5	引导加载程序
Kernel	145	6	150	内核
可用空间	20009	151	20159	可用空间

因此在磁盘划分 8000 个扇区作为交换区 (即 1000 页)，磁盘扇区划分如下:

```
1 #define MBR_SECTOR 0 // MBR扇区
2 #define BOOTLOADER_START_SECTOR 1 // Bootloader起始扇区
3 #define BOOTLOADER_SECTOR_COUNT 5 // Bootloader扇区数
4 #define KERNEL_START_SECTOR 6 // 内核起始扇区
5 #define KERNEL_SECTOR_COUNT 145 // 内核扇区数
6 #define SWAP_START_SECTOR 151 // 交换区起始扇区
7 #define SWAP_SECTOR_COUNT 8000 // 交换区扇区数
8 #define SWAP_END_SECTOR 8151 // 交换区结束扇区
9 #define TOTAL_SECTORS 20160 // 磁盘总扇区数
```

Listing 1: 磁盘扇区规划

2.2 磁盘 IO 操作

磁盘 IO 采用 LBA 读取方式，具体内容在 Lab3 中已经提及并实现，在此封装磁盘 IO 类

```
1 class DiskIO
2 {
3 public:
4     DiskIO();
5     bool readSector(uint32 lba, void* buffer);
6     bool writeSector(uint32 lba, const void* buffer);
7     // 检查LBA是否在交换区范围内
8     bool isSwapSector(uint32 lba);
9     // 获取交换区起始扇区
10    uint32 getSwapStartSector();
```

```
11 // 获取交换区扇区数量
12 uint32 getSwapSectorCount();
13
14 };
```

Listing 2: 磁盘 IO 类

交换区中的磁盘读/写的实现流程如下:

- 检查 LBA 是否在交换区范围内
- 等待磁盘准备就绪 (轮询状态位)
- 设置驱动器、扇区数量、LBA 地址
- 发送读/写命令
- 等待数据准备好
- 读取数据或写入数据

其中磁盘操作参考 Lab3 中的 LBA 读取方式, 读写硬盘的步骤如下

- **设置起始的逻辑扇区号:** LBA 的逻辑扇区号共 28 位, 其中 0-7 位写入 0x1F3 端口, 8-15 位写入 0x1F4 端口, 16-23 位写入 0x1F5 端口, 最后 4 位写入 0x1F6 端口
- 在 0x1F6 端口设置 LBA 方式和主硬盘
- 将要读取的扇区数量写入 0x1F2 端口
- 向 0x1F7 端口写入 0x20, 请求硬盘读
- 等待硬盘空闲
- 最后便能读写数据

上面涉及的端口, 我们用常量进行设置:

```
1 #define ATA_DATA_PORT 0x1F0
2 #define ATA_FEATURES_PORT 0x1F1
3 #define ATA_SECTOR_COUNT_PORT 0x1F2
4 #define ATA_LBA_LOW_PORT 0x1F3
5 #define ATA_LBA_MID_PORT 0x1F4
6 #define ATA_LBA_HIGH_PORT 0x1F5
7 #define ATA_DRIVE_HEAD_PORT 0x1F6
8 #define ATA_STATUS_PORT 0x1F7
9 #define ATA_COMMAND_PORT 0x1F7
10 #define ATA_CMD_READ_SECTORS 0x20
11 #define ATA_CMD_WRITE_SECTORS 0x30
12 #define ATA_STATUS_BSY 0x80 // 忙碌
13 #define ATA_STATUS_DRQ 0x08 // 数据请求
14 #define ATA_STATUS_ERR 0x01 // 错误
```

Listing 3: 磁盘常量

由于读写的实现逻辑相近, 在此展示读操作的实现:

```

1 bool DiskIO::readSector(uint32 lba, void* buffer)
2 {
3     if (!buffer) {
4         printf("Invalid buffer for disk read\n");
5         return false;
6     }
7     // 检查LBA是否有效
8     if (lba >= TOTAL_SECTORS) {
9         printf("Invalid LBA: %d (max: %d)\n", lba, TOTAL_SECTORS - 1);
10        return false;
11    }
12    uint8 status;
13    uint16* wordBuffer = (uint16*)buffer;
14    // 等待磁盘就绪
15    int timeout = 10000;
16    do {
17        asm_in_port(ATA_STATUS_PORT, &status);
18        timeout--;
19        if (timeout <= 0) {
20            printf("Disk read timeout waiting for ready\n");
21            return false;
22        }
23    } while (status & ATA_STATUS_BSY);
24    // 设置驱动器和LBA高4位 (LBA模式, 主磁盘, 以及逻辑扇区号24-27位)
25    asm_out_port(ATA_DRIVE_HEAD_PORT, 0xE0 | ((lba >> 24) & 0x0F));
26    // 设置扇区数量为1
27    asm_out_port(ATA_SECTOR_COUNT_PORT, 1);
28    // 设置LBA地址
29    asm_out_port(ATA_LBA_LOW_PORT, lba & 0xFF); //0-7位
30    asm_out_port(ATA_LBA_MID_PORT, (lba >> 8) & 0xFF); //8-15位
31    asm_out_port(ATA_LBA_HIGH_PORT, (lba >> 16) & 0xFF); //16-23位
32    // 发送读扇区命令, 即0x20到端口0x1F7
33    asm_out_port(ATA_COMMAND_PORT, ATA_CMD_READ_SECTORS);
34    // 等待数据准备好
35    timeout = 10000;
36    do {
37        asm_in_port(ATA_STATUS_PORT, &status);
38        timeout--;
39        if (timeout <= 0) {
40            printf("Disk read timeout waiting for data\n");
41            return false;
42        }
43    } while (!(status & ATA_STATUS_DRQ) && !(status & ATA_STATUS_ERR));
44    // 检查错误
45    if (status & ATA_STATUS_ERR) {
46        printf("Disk read error\n");
47        return false;
48    }
49    // 读取数据 (512字节 = 256个16位字)
50    for (int i = 0; i < 256; i++) {
51        asm_in_port_word(ATA_DATA_PORT, &wordBuffer[i]);
52    }
53    return true;

```

54 }

Listing 4: 磁盘读操作

2.3 交换区的管理

由于交换区被划分为 1000 个页，需要对这些页进行管理，包括页的分配，以及物理页换入交换区，从交换区中把页换到物理内存。创建一个交换区管理类来维护

```

1 class SwapAreaManager
2 {
3     int totalSwapPages;
4     int usedSwapPages;
5     int freeSwapPages;
6     bool swapPages[1000]; //管理交换区每一页是否被分配
7     SpinLock swapLock;
8 public:
9     SwapAreaManager();
10    void initialize();
11    //页的换出
12    void swapOut(uint32 vaddr,uint32 pagenum); //vaddr:换到外存的页的起始虚拟地址, pagenum:交换区
        的编号
13    //页的换入
14    void swapIn(uint32 vaddr,uint32 pagenum);
15    int pagenum_to_lba(uint32 pagenum); //页号转换为LBA编号(起始连续八个)
16    int allocateSwapPage(); //分配一个交换区页, 返回pagenum
17 };

```

Listing 5: 交换区管理类

实现如下，其中分配交换区时要上锁，保证互斥访问。还要注意一个页面 (4KB) 对应 8 个扇区 (8*512B)

```

1 void SwapAreaManager::initialize() {
2     totalSwapPages = SWAP_SECTOR_COUNT / 8+1;
3     usedSwapPages = 0;
4     freeSwapPages = totalSwapPages;
5     for (int i = 0; i < totalSwapPages; i++) {
6         swapPages[i] = false;
7     }
8 }
9 int SwapAreaManager::pagenum_to_lba(uint32 pagenum) {
10    return SWAP_START_SECTOR + (pagenum-1) * 8;
11 }
12 int SwapAreaManager::allocateSwapPage() {
13     swapLock.lock();
14     for (int i = 1; i < totalSwapPages; i++) {
15         if (!swapPages[i]) {
16             swapPages[i] = true;
17             usedSwapPages++;
18             freeSwapPages--;
19             swapLock.unlock();
20             return i;
21         }
22     }
23     swapLock.unlock();

```

```

24     return -1;
25 }
26 void SwapAreaManager::swapOut(uint32 vaddr, uint32 pagenum) {
27     uint32 lba_start = pagenum_to_lba(pagenum);
28     uint32 lba_end = lba_start + 8;
29     uint32 lba = lba_start;
30     while (lba < lba_end) {
31         diskIO.writeSector(lba, (void*)vaddr);
32         lba++;
33         vaddr += 512;
34     }
35 }
36 void SwapAreaManager::swapIn(uint32 vaddr, uint32 pagenum) {
37     uint32 lba_start = pagenum_to_lba(pagenum);
38     uint32 lba_end = lba_start + 8;
39     uint32 lba = lba_start;
40     while (lba < lba_end) {
41         diskIO.readSector(lba, (void*)vaddr);
42         lba++;
43         vaddr += 512;
44     }
45     swapLock.lock();
46     swapPages[pagenum] = false;
47     usedSwapPages--; //对这俩变量要互斥访问
48     freeSwapPages++;
49     swapLock.unlock();
50 }

```

Listing 6: 交换区的管理

2.4 开启缺页中断

按照 Lab4-中断的方法，注册并开启缺页中断：

首先在中断处理器中，注册缺页中断：

```

1 setInterruptDescriptor(14, (uint32)asm_page_fault_handler, 0);

```

Listing 7: 注册缺页中断

然后在 asm 中执行中断处理函数，这里要注意，缺页中断与时钟中断的不同点在于它还回返回一个中断的错误码在栈中，返回前需要进行弹栈弹出错误码：

```

1 asm_page_fault_handler:
2     pushad
3     call c_page_interrupt_handler
4     popad
5     ; 处理错误码
6     add esp, 4
7     iret

```

2.5 虚拟页的分配

分配虚拟页时采用延迟分配物理页的方法，不立即分配物理页，而是等到访问对应的虚拟地址时，通过缺页中断再进行物理页的分配。

```

1 int MemoryManager::allocatePages(enum AddressPoolType type, const int count)
2 {
3     int virtualAddress = allocateVirtualPages(type, count);
4     if (!virtualAddress)
5     {
6         return 0;
7     }
8
9     return virtualAddress;
10 }

```

Listing 8: 虚拟页的分配

2.6 请求页表的处理与页面的调出

如何标识一个页面是否在外存中，以及在外存中的哪个位置呢？可以从页表入手：

若此页面被换出到了磁盘交换区，则此页表的 P 位为 0，而原先存放物理地址的 31-12 位存放交换区的页号 (1-1000)，这里特意设交换区的起始页号为 1 而不是 0，从而可以从页表的 0 判断出此页面是从未出现过的 (不存在外存中)。

因此在 MemoryManager 中实现一个将页面换出到外存的操作，要注意此操作是原子的，需要加锁。

```

1 void MemoryManager::swapOut(uint32 vaddr) {
2     swap_lock.lock();
3     int pagenum = swapAreaManager.allocateSwapPage(); //分配交换区页
4     if (pagenum == -1) {
5         swap_lock.unlock();
6         printf("No swap space available\n");
7         return;
8     }
9     swapAreaManager.swapOut(vaddr, pagenum); //写到外存
10    int *pte = (int *)toPTE(vaddr);
11    memset((void *) (vaddr & 0xfffff000), 0, PAGE_SIZE);
12    flush_tlb_single(vaddr);
13    *pte &= 0xffffffe; //存在位为0
14    *pte &= 0x00000fff; //31-12位清空
15    *pte |= pagenum << 12; //31-12位设为pagenum
16    swap_lock.unlock();
17 }

```

Listing 9: 将页面调出到外存

2.7 虚拟页的释放

为了完善基于请求调页下的内存释放，需要考虑以下情况

- 释放内存时，此虚拟页还未分配物理页，即这个页对应的虚拟地址还没被访问过
- 释放内存时，此虚拟页已分配物理页，物理页在内存中
- 释放内存时，此虚拟页已分配物理页，但这个物理页被置换到了外存中。若直接释放，则在外存中的物理页会残余。

为了统一三种情况，在此采用统一方法：先访问一次这个页的虚拟地址，若此虚拟页为分配物理页或者物理页在外存，通过这次访问就会建立连接并调入内存，便可以使用我们原先的页释放方法。

```

1 void MemoryManager::releasePages(enum AddressPoolType type, const int virtualAddress, const int
   count)
2 {
3     int vaddr = virtualAddress;
4     *((int *)vaddr)=0; //为它调入物理页或者创建物理页
5     int *pte;
6     for (int i = 0; i < count; ++i, vaddr += PAGE_SIZE)
7     {
8         // 第一步, 对每一个虚拟页, 释放为其分配的物理页
9         releasePhysicalPages(type, vaddr2paddr(vaddr), 1);
10        // 设置页表项为不存在, 防止释放后被再次使用
11        pte = (int *)toPTE(vaddr);
12        *pte = 0;
13    }
14
15    // 第二步, 释放虚拟页
16    releaseVirtualPages(type, virtualAddress, count);
17 }

```

Listing 10: 页面释放

2.8 缺页中断处理

发生缺页中断时, 中断发生所在的虚拟地址会存入寄存器 CR2, 因此我们在中断处理时可通过 CR2 寄存器获取发生中断的虚拟地址:

```

1 uint32 read_cr2() {
2     uint32 value;
3     asm volatile("mov %%cr2, %0" : "=r"(value));
4     return value;
5 }

```

Listing 11: 获取发生缺页中断的虚拟地址

系统会自动给访问过的页面进行 TLB 缓存, 因此在释放页面之前要先刷新被置换掉的页面对应的 TLB

```

1 static inline void flush_tlb_single(int addr) {
2     asm volatile("invlpg (%0)" :: "r" (addr) : "memory");
3 }

```

Listing 12: 清空 TLB 缓存

约定只有手动分配过的虚拟地址才是合法的虚拟地址, 否则定义此地址不合法, 因此在缺页中断处理步骤:

- 从 CR2 寄存器读取虚拟地址
- 判断虚拟地址是否合法 (是否分配过)
- 为此虚拟地址分配物理页
- 若可分配的物理页已满, 则进行页面置换
- 由 FIFO 算法取出要换走的页, 先换到外存 (即把内容写入外存交换区)
- 将新分配的物理页加入队列
- 若此虚拟地址已含页表, 提取页表项中的 31-12 位, 为 pagenum

- 将虚拟页与物理页的建立页目录表、页表联系
- 若 pagenum 不为 0，说明此页面存在于交换区中，则需要从交换区中对应位置读入新页

中断处理函数如下

```

1 extern "C" void c_page_interrupt_handler()
2 {
3     uint32 cr2=read_cr2();
4     printf("page fault on virtual address :0x%x\n",cr2);
5     bool is_valid = false;
6
7     // 计算地址所在的页面
8     uint32 page_addr = cr2 & 0xFFFFF000;
9
10    // 检查是否是内核地址空间 (3GB 以上)
11    if (cr2 >= 0xC0000000) {
12        // 内核地址空间, 检查该地址是否在内核虚拟地址池的已分配范围内
13        int page_index = (page_addr - memoryManager.kernelVirtual.startAddress) / 4096;
14        if (page_index >= 0 && page_index < memoryManager.kernelVirtual.resources.length) {
15            // 检查该页是否已分配 (即位图中该位是否为1)
16            is_valid = memoryManager.kernelVirtual.resources.get(page_index);
17        }
18    } else {
19        is_valid = false;
20    }
21
22    if (!is_valid) {
23        printf("Illegal memory access at 0x%x, process terminated\n", cr2);
24        asm_halt();
25        return;
26    }
27
28    // 合法地址, 为其分配物理页
29    int physicalPageAddress;
30    bool flag=true;
31    physicalPageAddress = memoryManager.allocatePhysicalPages(AddressPoolType::KERNEL, 1);
32    if (physicalPageAddress == 0) {
33        flag=false;
34    }
35
36    if (!flag) { //物理页不足, 需要换出
37        if (memoryManager.physicalPageQueue.empty()) {
38            asm_halt();
39        }
40        item u=memoryManager.physicalPageQueue.getFront();
41        memoryManager.physicalPageQueue.pop();
42        memoryManager.swapOut(u.virtualAddress); //换到外存
43        memoryManager.releasePhysicalPages(AddressPoolType::KERNEL, u.physicalAddress, 1);
44        physicalPageAddress=memoryManager.allocatePhysicalPages(AddressPoolType::KERNEL, 1);
45        flush_tlb_single(u.virtualAddress);
46    }
47    memoryManager.physicalPageQueue.push(item(cr2,physicalPageAddress));
48    int *pte=(int *)memoryManager.toPTE(cr2);
49    int *pde=(int *)memoryManager.toPDE(cr2);

```

```
50     int pagenum;
51     if (!(*pde & 0x00000001)) pagenum=0;    //无对应页表
52         else pagenum=(*pte)>>12;    //有对应页表
53     flag=memoryManager.connectPhysicalVirtualPage(cr2, physicalPageAddress);
54     if (pagenum!=0) {
55         memoryManager.swapAreaManager.swapIn(cr2,pagenum);
56         printf("swap in page from disk\n");
57     }
58     if (!flag) {
59         printf("connect with physical page failed\n");
60         asm_halt();
61     }
62
63     printf("Connect virtual Page:%d with physical Page:%d\n",memoryManager.virtualPageID(cr2),
64         memoryManager.physicalPageID(physicalPageAddress));
65 }
```

Listing 13: 中断处理函数

2.9 测试样例的设计和运行结果

为了更符合请求调页的场景，以及便于设计测试样例，在此设置物理页框的数目为 5

```
1 int kernelphysicalpage=5;
```

Listing 14: MemoryManager 的 initialize 函数中分配驻留集

动态分配 8 个页面，并依次往页面中写入内容

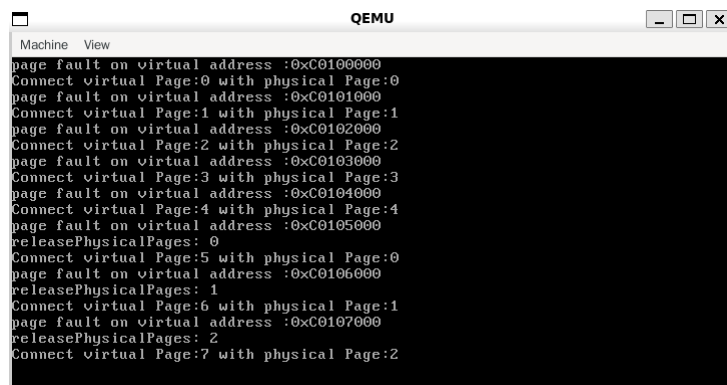
```
1 char *p=(char *)memoryManager.allocatePages(AddressPoolType::KERNEL,8);
2 char num='a';
3 for (int i=0;i<8;i++)
4 {
5     p[i*PAGE_SIZE]=num;
6     num++;
7     flush_tlb_single((int)p+i*PAGE_SIZE);
8 }
```

Listing 15: 页面分配

此时每个页面的状态如下

页面	0	1	2	3	4	5	6	7
首地址写入内容	a	b	c	d	e	f	g	h
所在位置	交换区	交换区	交换区	页框 3	页框 4	页框 0	页框 1	页框 2

先查看此时的输出，验证 FIFO 算法以及缺页中断处理的正确性:



```

Machine View
page fault on virtual address :0xC0100000
Connect virtual Page:0 with physical Page:0
page fault on virtual address :0xC0101000
Connect virtual Page:1 with physical Page:1
page fault on virtual address :0xC0102000
Connect virtual Page:2 with physical Page:2
page fault on virtual address :0xC0103000
Connect virtual Page:3 with physical Page:3
page fault on virtual address :0xC0104000
Connect virtual Page:4 with physical Page:4
page fault on virtual address :0xC0105000
releasePhysicalPages: 0
Connect virtual Page:5 with physical Page:0
page fault on virtual address :0xC0106000
releasePhysicalPages: 1
Connect virtual Page:6 with physical Page:1
page fault on virtual address :0xC0107000
releasePhysicalPages: 2
Connect virtual Page:7 with physical Page:2

```

可以看到缺页中断的处理和 FIFO 算法处理正确

接着我们对原先在交换区的页进行访问，看看能不能读到它们里面先前写入的，即检查页面的调出调入逻辑是否正确

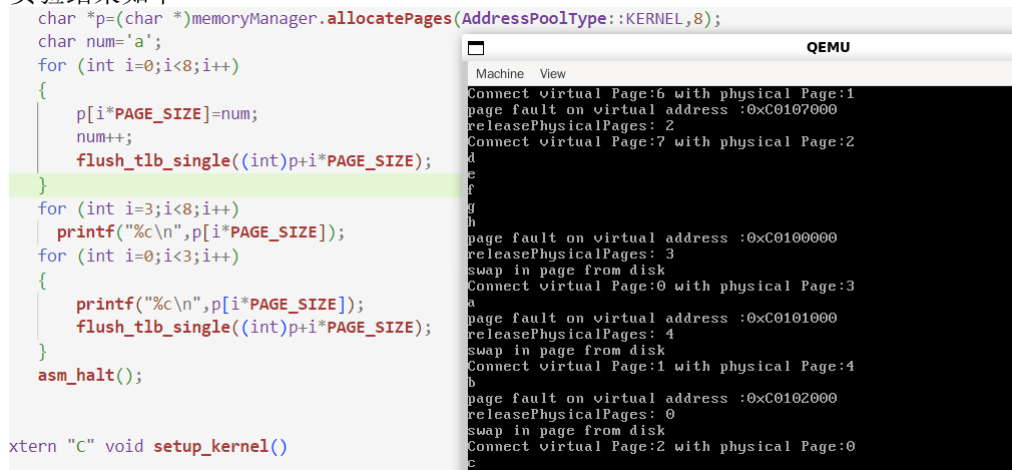
```

1 for (int i=0;i<3;i++)
2 {
3     printf("%c\n",p[i*PAGE_SIZE]);
4     flush_tlb_single((int)p+i*PAGE_SIZE);
5 }

```

Listing 16: 测试页面与外存的交换

实验结果如下



```

char *p=(char *)memoryManager.allocatePages(AddressPoolType::KERNEL,8);
char num='a';
for (int i=0;i<8;i++)
{
    p[i*PAGE_SIZE]=num;
    num++;
    flush_tlb_single((int)p+i*PAGE_SIZE);
}
for (int i=3;i<8;i++)
    printf("%c\n",p[i*PAGE_SIZE]);
for (int i=0;i<3;i++)
{
    printf("%c\n",p[i*PAGE_SIZE]);
    flush_tlb_single((int)p+i*PAGE_SIZE);
}
asm_halt();

extern "C" void setup_kernel()

```

```

Machine View
Connect virtual Page:6 with physical Page:1
page fault on virtual address :0xC0107000
releasePhysicalPages: 2
Connect virtual Page:7 with physical Page:2
page fault on virtual address :0xC0100000
releasePhysicalPages: 3
swap in page from disk
Connect virtual Page:0 with physical Page:3
page fault on virtual address :0xC0101000
releasePhysicalPages: 4
swap in page from disk
Connect virtual Page:1 with physical Page:4
page fault on virtual address :0xC0102000
releasePhysicalPages: 0
swap in page from disk
Connect virtual Page:2 with physical Page:0

```

可以看到，成功读取到正确内容，说明成功实现了物理页框与外存的调入调出逻辑。

3 Project1 到 Project2 的过渡: 从内核态到用户态

为了方便对样例进行测试，Project1 采用的是基于 Lab7 以及之前所建立的内核态的代码体系。在 Lab8 中实现了从内核态到用户态的转变，在此将 Project1 中关于虚拟内存以及外存的管理融入到 Lab8 中涉及用户态的代码中。

由于用户态的加入，需要考虑的主要问题为:

1. 在缺页中断中，中断对应的地址应该是内核地址还是用户地址？
2. 页面置换中，应该换的是用户空间的物理页还是内核空间的物理页？

首先，不同的地址要对应使用不同状态的地址池进行物理内存分配。

其次，还要用各自的方法判断访问的地址到底合不合法 (是否是 allocate 过的虚拟地址): 如果是内核地址，则通过内核的虚拟地址池的位图判断，如果是用户地址，则通过用户进程的虚拟地址池的位图判断。

最后，由于用户空间的虚拟地址对应物理页由用户地址池维护，内核空间的物理页由内核地址池来维护，因此在处理的时候要加以区分

因此，首先对内存管理类进行修改，区分内核物理页队列和用户物理页队列：

```

1 class MemoryManager
2 {
3 public:
4     int totalMemory;
5     AddressPool kernelPhysical;
6     AddressPool userPhysical;
7     AddressPool kernelVirtual;
8
9     Queue KernelphysicalPageQueue;    //内核物理页队列
10    Queue UserphysicalPageQueue;      //用户物理页队列
11
12    int kernelPhysicalStartAddress;
13    int userPhysicalStartAddress;
14    Free_queue free_queue;    //后续的 malloc 和 free 会用到
15    SwapAreaManager swapAreaManager;

```

Listing 17: 内存管理类

接着对中断处理函数进行更改：

首先，获取中断地址

```

1 extern "C" void c_page_interrupt_handler()
2 {
3     uint32 cr2=read_cr2();
4     printf("page fault on virtual address :0x%x\n",cr2);
5
6     // 计算地址所在的页面
7     uint32 page_addr = cr2 & 0xFFFFF000;

```

Listing 18: 中断处理函数：获取中断地址

接着，根据内核态和用户态，判断此中断地址是否是已分配的合法地址：

```

1 bool is_valid = false;
2 if (cr2 >= KERNEL_VIRTUAL_START) {
3     int page_index = (page_addr - memoryManager.kernelVirtual.startAddress) / 4096;
4     if (page_index >= 0 && page_index < memoryManager.kernelVirtual.resources.length) {
5         // 检查该页是否已分配（即位图中该位是否为1）
6         is_valid = memoryManager.kernelVirtual.resources.get(page_index);
7     }
8 } else if (cr2 >= USER_VADDR_START && cr2 < 0xc0000000) {
9     PCB* cur = programManager.running;
10    int page_index = (page_addr - cur->userVirtual.startAddress) / 4096;
11    if (page_index >= 0 && page_index < cur->userVirtual.resources.length) {
12        is_valid = cur->userVirtual.resources.get(page_index);
13    }
14 } else {
15     is_valid = true;
16 }

```

```

17 if (!is_valid) {
18     printf("Illegal memory access at 0x%x, process terminated\n", cr2);
19     asm_halt();
20     return;
21 }

```

Listing 19: 中断处理函数：判断地址是否合法

对于合法地址，分配物理页，要区分用户态和内核态：

```

1 // 合法地址，为其分配物理页
2 int physicalPageAddress;
3 bool flag=true;
4 if (cr2 >= 0xC0000000) {
5     physicalPageAddress = memoryManager.allocatePhysicalPages(AddressPoolType::KERNEL, 1);
6 } else {
7     physicalPageAddress = memoryManager.allocatePhysicalPages(AddressPoolType::USER, 1);
8 }
9 if (physicalPageAddress == 0) {
10     flag=false;
11 }

```

Listing 20: 中断处理函数：分配物理页

物理页不足，进行换出，释放和分配物理页需要区分用户态和内核态考虑

```

1 if (!flag) {
2     if (cr2>=KERNEL_VIRTUAL_START) { //内核态
3         if (memoryManager.KernelphysicalPageQueue.empty()) {
4             asm_halt();
5         }
6         item u=memoryManager.KernelphysicalPageQueue.getFront();
7         memoryManager.KernelphysicalPageQueue.pop();
8         memoryManager.swapOut(u.virtualAddress);
9         memoryManager.releasePhysicalPages(AddressPoolType::KERNEL, u.physicalAddress, 1);
10        physicalPageAddress=memoryManager.allocatePhysicalPages(AddressPoolType::KERNEL, 1);
11        flush_tlb_single(u.virtualAddress);
12    }
13    else if (cr2>=USER_VADDR_START&&cr2<0xc0000000) { //用户态
14        if (memoryManager.UserphysicalPageQueue.empty()) {
15            asm_halt();
16        }
17        item u=memoryManager.UserphysicalPageQueue.getFront();
18        memoryManager.UserphysicalPageQueue.pop();
19        memoryManager.swapOut(u.virtualAddress);
20        memoryManager.releasePhysicalPages(AddressPoolType::USER, u.physicalAddress, 1);
21        physicalPageAddress=memoryManager.allocatePhysicalPages(AddressPoolType::USER, 1);
22        flush_tlb_single(u.virtualAddress);
23    }
24 }
25 }
26 if (cr2>=KERNEL_VIRTUAL_START) {
27     memoryManager.KernelphysicalPageQueue.push(item(cr2,physicalPageAddress));
28 }
29 else if (cr2>=USER_VADDR_START&&cr2<0xc0000000) {
30     memoryManager.UserphysicalPageQueue.push(item(cr2,physicalPageAddress));

```

31 }

Listing 21: 中断处理函数：页面置换

最后建立虚拟页和物理页的联系，外存有对应页时则调入

```

1  memoryManager.physicalPageQueue.push(item(cr2,physicalPageAddress));
2  int *pte=(int *)memoryManager.toPTE(cr2);
3  int *pde=(int *)memoryManager.toPDE(cr2);
4  int pagenum;
5  if (!(*pde & 0x00000001)) pagenum=0;    //无对应页表
6      else pagenum=(*pte)>>12;    //有对应页表
7  flag=memoryManager.connectPhysicalVirtualPage(cr2, physicalPageAddress);
8  if (pagenum!=0) {
9      memoryManager.swapAreaManager.swapIn(cr2,pagenum);
10     printf("swap in page from disk\n");
11 }
12 if (!flag) {
13     printf("connect with physical page failed\n");
14     asm_halt();
15 }
16
17 }
```

Listing 22: 中断处理函数：建立虚拟页物理页联系

4 Project2:malloc 和 free

本项目为在虚拟内存的页面管理的基础上，实现 malloc 和 free 操作。其中 malloc 和 free 函数用于用户态进程中的动态内存分配与回收。

4.1 用户态的内存分配

在之前的实验中，我们实现了内核线程，并可以在内核线程中通过

```

1 char *p=(char *)memoryManager.allocatePages(AddressPoolType::KERNEL,8);
```

来进行内存分配。但 memoryManager 是内核级的内存控制类，在用户态中直接进行使用是不合适的。因此要想对用户态的用户进程进行动态内存分配，就先要封装系统调用，在此 Project 中体现为封装 malloc 和 free。malloc 用于动态分配指定字节大小的内存，free 用于回收 malloc 分配的内存。

首先封装系统调用

```

1 // 设置5号系统调用
2 systemService.setSystemCall(5, (int)syscall_malloc);
3 // 设置6号系统调用
4 systemService.setSystemCall(6, (int)syscall_free);
5 void *syscall_malloc(int size) {
6     return memoryManager.malloc(size);
7 }
8 void *malloc(int size) {    //malloc:分配size字节内存，返回指向内存起始地址的void*指针
9     return (void *)asm_system_call(5, size);
10 }
11 void syscall_free(void *ptr,int size) {
```

```

12     memoryManager.free(ptr,size);
13 }
14 void free(void *ptr,int size) { //free:回收从*ptr开始的连续size字节内存
15     asm_system_call(6, (int)ptr,size);
16     ptr=nullptr;
17 }

```

Listing 23: 封装系统调用

4.2 空闲块的管理

现有的框架已经实现了以页面为单位的空间分配，而以字节为单位的 malloc 需要把每个页的 4KB 空间进行拆分，选择合适的动态内存分配算法和空闲空间管理方法。

一个页的大小为 4KB，涉及 4096 个块，若采用位图等数据结构对每一页的所有空间进行管理，则内存占用开销会很大。综合考虑，我采用以下方法管理每一个空闲块：

```

1 struct free_block {
2     int vaddr; //此空闲块起始虚拟地址
3     int size;  //此空闲块的大小
4     free_block(int vaddr,int size)
5     {
6         this->vaddr=vaddr;
7         this->size=size;
8     }
9     free_block(){}
10 };

```

Listing 24: 空闲块

而且只有一个页面在 malloc 的过程中被动态分配，这个页里才会产生空闲块。

在 malloc 和 free 的过程中，用一个队列管理空闲块

```

1 class Free_queue
2 {
3     public:
4         free_block queue[MAX_QUEUE_SIZE];
5         int front;
6         int rear;
7         Free_queue()
8         {
9             front=0;
10            rear=0;
11        }
12        void push(free_block value)
13        {
14            queue[rear]=value;
15            rear=(rear+1)%MAX_QUEUE_SIZE;
16        }
17        void pop()
18        {
19            front=(front+1)%MAX_QUEUE_SIZE;
20        }
21        free_block getFront()
22        {

```

```

23     return queue[front];
24 }
25 int size()
26 {
27     return (rear-front+MAX_QUEUE_SIZE)%MAX_QUEUE_SIZE;
28 }
29 void del(int index)
30 {
31     for (int i=index;i!=rear;i=(i+1)%MAX_QUEUE_SIZE)
32     {
33         queue[i]=queue[(i+1)%MAX_QUEUE_SIZE];
34     }
35     rear--;
36 }
37 };

```

Listing 25: 空闲块队列

并在内存管理类中加入空闲块管理队列

```

1 class MemoryManager
2 {
3 public:
4     int totalMemory;
5     AddressPool kernelPhysical;
6     AddressPool userPhysical;
7     AddressPool kernelVirtual;
8     Queue KernelphysicalPageQueue;
9     Queue UserphysicalPageQueue;
10    int kernelPhysicalStartAddress;
11    int userPhysicalStartAddress;
12    Free_queue free_queue; //空闲块队列
13    SwapAreaManager swapAreaManager;

```

Listing 26: 内存管理类

4.3 malloc 的实现

对于 malloc 的实现，规定如下：

1. malloc 分配的内存必须要在虚拟空间连续
2. 在分配空间连续的前提下，尽量减少页面的分配。

malloc 的实现过程如下：

- 若需要动态分配的内存大小小于一个页，先遍历空闲块队列
- 若存在空闲块的大小大于需要分配的内存大小，则按照最佳适应算法选择空闲块分配
- 分配后更新此空闲块的大小，若为 0 则移除
- 若需要动态分配的内存大小大于一个页或者不存在空闲块的大小大于需要分配的内存大小，则直接新分配页面
- 新分配页面的情况下，分配的最后一页 (可能) 会产生空闲块 (即这一页没用完)，将此空闲块加入空闲块队列
- 返回 void* 类型的分配内存的起始地址指针

代码如下，其中 malloc 对于各进程的原子性，必须要加锁和解锁

```

1 void *MemoryManager::malloc(int size)    //规定分配的空间必须连续
2 {
3     malloc_lock.lock();    //上锁
4     if (size<0) {
5         malloc_lock.unlock();
6         return nullptr;
7     }
8     int minn=1e9;
9     int index=-1;
10    if (size<PAGE_SIZE) {
11        for (int i=free_queue.front;i!=free_queue.rear;i=(i+1)%MAX_QUEUE_SIZE)
12        {
13            if (free_queue.queue[i].size>=size&&free_queue.queue[i].size<minn)
14            {
15                minn=free_queue.queue[i].size;
16                index=i;
17            }
18        }
19    }
20    if (index!=-1) {
21        int start=free_queue.queue[index].vaddr;
22        free_queue.queue[index].vaddr+=size;
23        free_queue.queue[index].size-=size;
24        if (free_queue.queue[index].size==0) {
25            free_queue.del(index);
26        }
27        memset((void *)start,0,size);
28        malloc_lock.unlock();
29        return (void *)start;
30    }
31    int start=allocatePages(AddressPoolType::USER,ceil(size,PAGE_SIZE));
32    if (start==0) {
33        malloc_lock.unlock();
34        return nullptr;
35    }
36    if (size%PAGE_SIZE!=0) {
37        int pos=start+(ceil(size,PAGE_SIZE)-1)*PAGE_SIZE;
38        pos+=size%PAGE_SIZE;
39        free_queue.push(free_block(pos,PAGE_SIZE-size%PAGE_SIZE));
40    }
41    memset((void *)start,0,size);
42    malloc_lock.unlock();
43    return (void *)start;
44 }

```

Listing 27: malloc

4.4 free 的实现

在 malloc 的基础上，free 的实现较为困难。与 c 语言类似，在此 free 函数不做内存检查，用户使用 free 函数时需要与 malloc 的行为保持一致，否则会产生未定义行为。

free 的实现过程如下：

- 若占用超过一页，则对整页的空间进行释放，并释放掉此页
- 若不足一页，则分四种情况
- 情况一：释放的这一块前后都没有相连的空闲块，则将此空闲块加入空闲队列
- 情况二：释放的这一块前面有相连空闲块后面没有，则进行合并，更新前面这一空闲块的大小
- 情况三：释放的这一块后面有相连空闲块前面没有，则进行合并，更新后面这一空闲块大小
- 情况四：释放的这一块前后都有相连的空闲块，则进行合并，删除后面这一空闲块，合并至前面这一空闲块
- 若合并的后的空闲块大小为一整页，则释放掉整页空间，也将此空闲块移除队列

具体代码逻辑如下，其中对空闲块的访问必须是原子的，要注意上锁与解锁

```

1 void MemoryManager::free(void *ptr,int size)
2 {
3     free_lock.lock();
4     //若size小于一页:
5     if (size<PAGE_SIZE) {
6         memset((void *)ptr,0,size);
7         int nextstart=(int)ptr+size;
8         int lastend=(int)ptr-1;
9         bool has_last=false;
10        bool has_next=false;
11        int next_index=-1;
12        int last_index=-1;
13        for (int i=free_queue.front;i!=free_queue.rear;i=(i+1)%MAX_QUEUE_SIZE)
14        {
15            if (free_queue.queue[i].vaddr==nextstart) {
16                free_queue.queue[i].vaddr=(int)ptr;
17                free_queue.queue[i].size+=size;
18                if (free_queue.queue[i].size==PAGE_SIZE) {
19                    releasePages(AddressPoolType::USER,free_queue.queue[i].vaddr,1);
20                    free_queue.del(i);
21                }
22                has_next=true;
23                next_index=i;
24            }
25            if (free_queue.queue[i].vaddr+free_queue.queue[i].size-1==lastend) {
26                has_last=true;
27                last_index=i;
28            }
29        }
30        if (has_next&&!has_last) return; //有后无前
31        if (has_last&&!has_next) { //有前无后
32            free_queue.queue[last_index].size+=size;
33            if (free_queue.queue[last_index].size==PAGE_SIZE) {
34                releasePages(AddressPoolType::USER,free_queue.queue[last_index].vaddr,1);
35                free_queue.del(last_index);
36            }
37        } else
38        if (has_next&&has_last) { //前后都有
39            free_queue.queue[last_index].size+=free_queue.queue[next_index].size;
40            free_queue.del(next_index);
41            for (int i=free_queue.front;i!=free_queue.rear;i=(i+1)%MAX_QUEUE_SIZE) {

```

```

42         if (free_queue.queue[i].size==PAGE_SIZE) {
43             releasePages(AddressPoolType::USER,free_queue.queue[i].vaddr,1);
44             free_queue.del(i);
45             break;
46         }
47     }
48     } else free_queue.push(free_block((int)ptr,size)); //前后都无
49     free_lock.unlock();
50     return;
51 }
52 //若size大于一页，则ptr一定位于一个页的开始
53 while (size)
54 {
55     if (size>=PAGE_SIZE) {
56         releasePages(AddressPoolType::USER,(int)ptr,1);
57         size-=PAGE_SIZE;
58         ptr+=PAGE_SIZE;
59     } else {
60         free_lock.unlock();
61         free(ptr,size);
62         return;
63     }
64 }
65 free_lock.unlock();
66 }

```

Listing 28: free

4.5 malloc & free 的简单样例测试

为了简单测试 malloc 和 free 的功能，尤其是 free 中的空闲块合并和页面释放部分，设置了此样例 (其中为了展示算法的正确性，使用了越权指令，在实际的用户进程中是不允许的)：

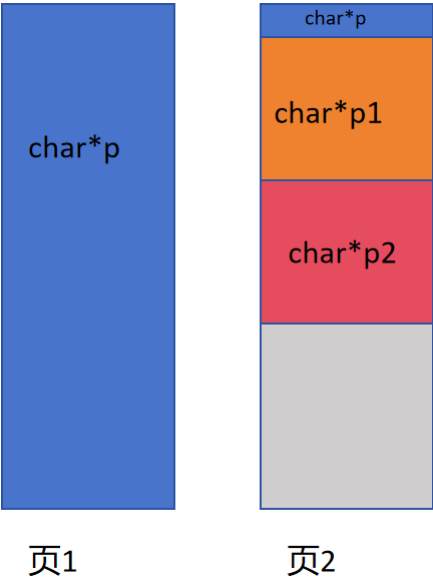
```

1 void first_process()
2 {
3     printf("first process\n");
4     char *p=(char *)malloc(4097);
5     printf("0x%x\n",p);
6     char *p1=(char *)malloc(1024);
7     printf("0x%x\n",p1);
8     char *p2=(char *)malloc(1024);
9     printf("0x%x\n",p2);
10    printf("number of free blocks: %d\n",memoryManager.free_queue.size());//这里是越权指令，不能使
    用(这里只是为了展示算法逻辑是否正确)
11    free(p,4097);
12    printf("number of free blocks: %d\n",memoryManager.free_queue.size());
13    free(p1,1024);
14    printf("number of free blocks: %d\n",memoryManager.free_queue.size());
15    free(p2,1024);
16    printf("number of free blocks: %d\n",memoryManager.free_queue.size());
17 }

```

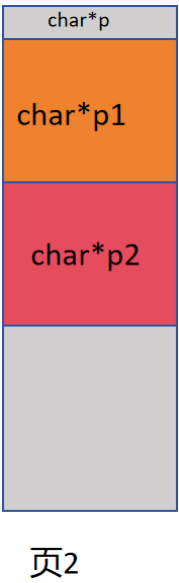
Listing 29: malloc 和 free 的测试样例

进行了前三个 malloc 后，内存的情况为:



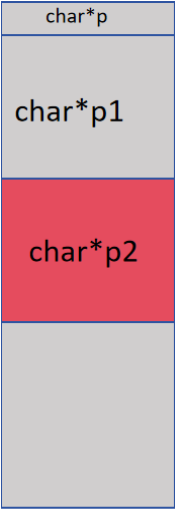
可以看到，此时共有一个空闲块 (页 2)

释放掉指针 p 指向的空间后，页 1 被释放，此时内存的情况为



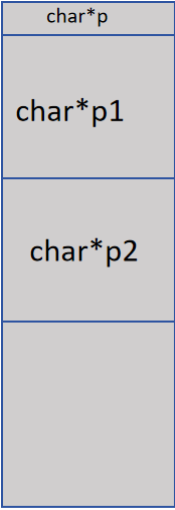
此时页 2 有两个空闲块

释放掉 p1 后，得到的空闲块要与上面直接相连的空闲块合并为一个新的空闲块:



页2

释放掉 p2 之后，得到的空闲块既要与上面直接相连的空闲块合并，也要与下面直接相连的空闲块合并，合并完后的空闲块刚好为一页，因此要释放掉这个空闲块和这个页



页2

因此，最后剩 0 个空闲块。

在上述测试样例的代码中，我在每次 free 后都查看了一次空闲块的个数，输出结果如下：

```
void first_process()
{
    printf("first process\n");
    char *p=(char *)malloc(4097);
    printf("0x%x\n",p);
    char *p1=(char *)malloc(1024);
    printf("0x%x\n",p1);
    char *p2=(char *)malloc(1024);
    printf("0x%x\n",p2);
    printf("number of free blocks: %d\n",memoryManager.free_queue.size());
    free(p,4097);
    printf("number of free blocks: %d\n",memoryManager.free_queue.size());
    free(p1,1024);
    printf("number of free blocks: %d\n",memoryManager.free_queue.size());
    free(p2,1024);
    printf("number of free blocks: %d\n",memoryManager.free_queue.size());
}
```

```
Machine View
first process
0x8049000
0x804a001
0x804a401
number of free blocks: 1
number of free blocks: 2
number of free blocks: 2
number of free blocks: 0
exit, pid: 1
release zombie process, pid: 1
```

通过此样例，malloc 和 free 的逻辑基本没问题。

5 从 Project2 到 Project5

5.1 Project2 存在的问题

Project2 在 project1 基础上将请求调页机制实现到了用户态，并实现了基于用户态的 malloc 和 free 函数。但是当我在 Project2 的线程函数执行多于一个 fork 函数时：

```
1 void first_process()
2 {
3     int pid1=fork();
4     int pid2=fork();
5 }
```

Listing 30: Project2 的 bug

执行 make run 后，qemu 竟然离奇的发生闪退。而当我删去第二个 fork，只执行一个 fork 时，程序是可以正常运行的。

面对此情况，我使用了 gdb 进行 debug，程序成功执行了第一个 fork。在父进程从第一个 fork 返回后程序进入到了第二个 fork 的执行，并执行到 copyProcess 函数的以下循环时，qemu 发生了闪退，即程序发生了崩溃：

```
1 for (int i = 0; i < 768; ++i)
2 {
3     // 无对应页表
4     if (!(parentPageDir[i] & 0x1))
5     {
6         continue;
7     }
8     // 从用户物理地址池中分配一页，作为子进程的页目录项指向的页表
9     int paddr = memoryManager.allocatePhysicalPages(AddressPoolType::USER, 1);
10    if (!paddr)
11    {
12        child->status = ProgramStatus::DEAD;
13        return false;
14    }
15    // 页目录项
16    int pde = parentPageDir[i];
17    // 构造页表的起始虚拟地址
18    int *pageTableVaddr = (int *) (0xffc00000 + (i << 12));
19    asm_update_cr3(childPageDirPaddr); // 进入子进程虚拟地址空间
20    childPageDir[i] = (pde & 0x00000fff) | paddr;
21    memset(pageTableVaddr, 0, PAGE_SIZE);
22    asm_update_cr3(parentPageDirPaddr); // 回到父进程虚拟地址空间
23 }
```

Listing 31: 程序崩溃的发生点

即子进程在拷贝页目录表和页表时 qemu 闪退了。

5.2 解决方案

经过查询资料，qemu 显示屏发生闪退的原因可能有以下几种：

- CPU 发生 Triple Fault, 页面错误无法正确处理、中断向量失效时，CPU 会执行第三次故障自动复位。(即 IDT 加载错误)

- 程序发生越权访问
- 内存空间、资源耗尽，程序发生未定义的异常

即使经过精细的 gdb 调试，也无法确定 qemu 闪退的真实原因。因为在 lab8 提供的 fork 示例中，是可以正常进行多个 fork 的，于是将此程序与 lab8 中提供的 fork 示例进行对比，两者只有如下差别：

lab8 中的虚拟页分配时是直接指定了物理页，而我的实现是在用户和内核态都延迟分配物理页，在访问时通过缺页中断进行分配

考虑到 fork 的实现中，子进程需要大幅拷贝父进程的资源，其中涉及了许多内核态资源的内存访问/拷贝（如页表、特权级 0 资源等），或许问题就出在内核页的访存上，发生了一些我没有遇见的错误。

因此我的解决方案如下，对于内核页的分配，直接指定物理页，而请求调页机制仅用于用户页。在页分配的时候要注意上锁和解锁：

```

1 int MemoryManager::allocatePages(enum AddressPoolType type, const int count)
2 {
3     allocate_lock.lock();
4     if (type==AddressPoolType::USER) {
5         int virtualAddress = allocateVirtualPages(AddressPoolType::USER, count);
6         if (!virtualAddress)
7         {
8             allocate_lock.unlock();
9             return 0;
10        }
11        allocate_lock.unlock();
12        return virtualAddress;
13    }
14    // 第一步：从虚拟地址池中分配若干虚拟页
15    int virtualAddress = allocateVirtualPages(AddressPoolType::KERNEL, count);
16    if (!virtualAddress)
17    {
18        allocate_lock.unlock();
19        return 0;
20    }
21
22    bool flag;
23    int physicalPageAddress;
24    int vaddress = virtualAddress;
25
26    // 依次为每一个虚拟页指定物理页
27    for (int i = 0; i < count; ++i, vaddress += PAGE_SIZE)
28    {
29        flag = false;
30        // 第二步：从物理地址池中分配一个物理页
31        physicalPageAddress = allocatePhysicalPages(AddressPoolType::KERNEL, 1);
32        if (physicalPageAddress)
33        {
34            //printf("allocate physical page 0x%x\n", physicalPageAddress);
35
36            // 第三步：为虚拟页建立页目录项和页表项，使虚拟页内的地址经过分页机制变换到物理页内。
37            flag = connectPhysicalVirtualPage(vaddress, physicalPageAddress);
38        }
39        else

```

```

40     {
41         flag = false;
42     }
43
44     // 分配失败，释放前面已经分配的虚拟页和物理页表
45     if (!flag)
46     {
47         // 前 i 个页表已经指定了物理页
48         releasePages(AddressPoolType::KERNEL, virtualAddress, i);
49         // 剩余的页表未指定物理页
50         releaseVirtualPages(AddressPoolType::KERNEL, virtualAddress + i * PAGE_SIZE, count - i)
51         ;
52         allocate_lock.unlock();
53         return 0;
54     }
55
56     allocate_lock.unlock();
57     return virtualAddress;
58 }

```

Listing 32: allocatePages 函数

如此，经过测试，多重 fork 的问题正式解决。为 project5 的写时复制做好了准备。

6 Project5: 写时复制 (COW)

写时复制，即为了减小资源的开销，fork 时子进程会先与父进程共享物理页，当父/子进程需要修改共享内容的时候重新分配新页，确保进程之间的数据隔离。

6.1 没有写时复制的情况

首先定义一个访问当前虚拟地址对应物理页地址的函数

```

1 uint32 getpaddr(uint32 vaddr)
2 {
3     int *pte=(int *)memoryManager.toPTE(vaddr);
4     return *pte&0xfffff000;
5 }

```

Listing 33: 访问物理页地址

在 Project2 的程序中，进行如下测试：先在父进程 malloc 一个共享变量，接着在下面进行 fork 分叉。然后父子进程分别对共享变量进行写，然后在访问它的虚拟地址、物理地址、值。

```

1 void second_process()
2 {
3     printf("second process\n");
4     int retval;
5     char *p=(char *)malloc(15);
6     strcpy("hello",p);
7     int pid=fork();
8     if (pid==0) {
9         printf("child process  vaddr:%x paddr:%x %s\n",p,getpaddr((int)p),p);
10        strcpy("world",p);

```



```

11     printf("child process  vaddr:%x paddr:%x %s\n\n",p,getpaddr((int)p),p);
12 } else {
13     printf("parent process\n");
14     printf("parent process  vaddr:%x paddr:%x %s\n",p,getpaddr((int)p),p);
15     strcpy("sysu",p);
16     printf("parent process  vaddr:%x paddr:%x %s\n",p,getpaddr((int)p),p);
17 }
18 asm_halt(); //防止p被释放(在这里没用)
19 }

```

Listing 34: 小测试

结果为



可得到以下结论:

- 父子进程中共享变量的虚拟地址是相同的，但通过两者不同的页目录表 → 页表映射，映射到了不同的物理地址
- 写操作的前后，p 指向的区域的物理地址没有改变，且写操作前父子进程的物理地址不相同，即不符合写时复制的情况。

因此 Project5 要做的就是，通过写时复制 (COW) 机制，使得父子进程在对共享区域进行写操作前，此虚拟地址映射到相同的物理页上，而任一进程写操作后此虚拟地址就映射到了不同物理页中。

6.2 写时复制 (COW) 应考虑的几个问题

1. 父子进程应共享什么资源，而应独享什么资源？页目录表和页表需要共享吗？所有物理页都能共享吗？
2. 写时复制机制应该怎么实现？即进程写访问时，该如何知道这个页是否需要另外复制？
3. 假如共享一个页的多个进程同时对这个页进行了写操作，根据 COW 机制，它们都会新创建一个物理页重新映射，那原本这个共享物理页就“没有主人”了？
4. 假设一个物理页原本有多个进程（父又生子，子又生孙，子子孙孙无穷匮也）共享，如果其中若干个进程结束，调用 exit 函数时删去了这个物理页，其它进程怎么办？否则，该如何知道何时删去这一页？

6.3 父子进程应共享什么资源

关于页目录表和页表的处理，子进程必须复制父进程的页目录/页表，只在页级别共享数据。

页表（和页目录）本质上是控制结构，决定虚拟地址如何映射到物理内存，以及该页是否可写、可读等权限。

- 如果父子进程直接共享页表或页目录指针，它们就共享了页表结构本身，也就是说父子进程看到的是完全相同的地址映射结构体。
- 如果一个进程要修改页表（比如写入时触发 COW，要取消只读并重新映射），那会影响另一个进程的页表结构，这是极其危险的。

因此，在 copyProcess 的第一个循环需要保留：

```

1 for (int i = 0; i < 768; ++i)
2 {
3     // 无对应页表

```

```

4     if (!(parentPageDir[i] & 0x1))
5     {
6         continue;
7     }
8     // 从用户物理地址池中分配一页，作为子进程的页目录项指向的页表
9     int paddr = memoryManager.allocatePhysicalPages(AddressPoolType::USER, 1);
10    if (!paddr)
11    {
12        child->status = ProgramStatus::DEAD;
13        return false;
14    }
15    // 页目录项
16    int pde = parentPageDir[i];
17    // 构造页表的起始虚拟地址
18    int *pageTableVaddr = (int *) (0xffc00000 + (i << 12));
19
20    asm_update_cr3(childPageDirPaddr); // 进入子进程虚拟地址空间
21
22    childPageDir[i] = (pde & 0x00000fff) | paddr;
23    memset(pageTableVaddr, 0, PAGE_SIZE);
24
25    asm_update_cr3(parentPageDirPaddr); // 回到父进程虚拟地址空间
26 }

```

Listing 35: copyProcess 函数：深拷贝父进程的页目录表和页表

第二个问题是，父子进程哪些页能共享，哪些页不能共享？

用户空间共有 768 页，如果我让父子进程共享所有用户页，则会发生一些不可预料的错误。因为在程序中我们定义了

```
1 #define USER_VADDR_START 0x8048000
```

Listing 36: 用户虚拟空间起始地址

而位于 0x8048000 以下的地址为一些与内核空间相关、内核管理相关的地址段，直接共享这段地址可能会发生补课预料的错误，因此不共享这段地址。

因此只共享 0x8048000 以上的地址，为了方便，父子进程从第 33 个页目录号开始的页进行共享。

最终的 copyProcess 中物理页的共享设定代码如下：

```

1 for (int i = 0; i < 33; ++i)
2 {
3     // 无对应页表
4     if (!(parentPageDir[i] & 0x1))
5     {
6         continue;
7     }
8
9     // 计算页表的虚拟地址
10    int *pageTableVaddr = (int *) (0xffc00000 + (i << 12));
11
12    // 复制物理页
13    for (int j = 0; j < 1024; ++j)
14    {
15        // 无对应物理页

```

```

16     if (!(pageTableVaddr[j] & 0x1))
17     {
18         continue;
19     }
20
21     // 从用户物理地址池中分配一页，作为子进程的页表项指向的物理页
22     int paddr = memoryManager.allocatePhysicalPages(AddressPoolType::USER, 1);
23     if (!paddr)
24     {
25         child->status = ProgramStatus::DEAD;
26         return false;
27     }
28
29     // 构造物理页的起始虚拟地址
30     void *pageVaddr = (void *)((i << 22) + (j << 12));
31     memcpy(pageVaddr, buffer, PAGE_SIZE);
32     // 页表项
33     int pte = pageTableVaddr[j];
34
35     asm_update_cr3(childPageDirPaddr); // 进入子进程虚拟地址空间
36
37     pageTableVaddr[j] = (pte & 0x00000fff) | paddr;
38     memcpy(buffer, pageVaddr, PAGE_SIZE);
39
40     asm_update_cr3(parentPageDirPaddr); // 回到父进程虚拟地址空间
41 }
42 }
43 for (int i=33;i<768;i++) //从0x08400000开始，父子共享
44 {
45     // 无对应页表
46     if (!(parentPageDir[i] & 0x1))
47     {
48         continue;
49     }
50
51     // 计算页表的虚拟地址
52     int *pageTableVaddr = (int *) (0xffc00000 + (i << 12));
53
54     for (int j=0;j<1024;j++)
55     {
56         if (!(pageTableVaddr[j] & 0x1))
57         {
58             continue;
59         }
60         pageTableVaddr[j]&=0xffffffff; //设置为只读
61         int pte = pageTableVaddr[j];
62         int paddr=pte&0xfffff000; //设置页表项只读
63         int index=memoryManager.read_only_page_queue.find_page(paddr); //处理物理页引用
64         if (index==-1) memoryManager.read_only_page_queue.add_page(paddr);
65         asm_update_cr3(childPageDirPaddr);
66         memoryManager.read_only_page_queue.add_page(paddr);
67         pageTableVaddr[j]=pte;
68         asm_update_cr3(parentPageDirPaddr);
69     }

```

70

}

Listing 37: copyProcess: 物理页共享

6.4 写时复制机制的实现

写时复制的实现方法是：将共享的物理页的页表项设置为只读，当父子进程对只读页面进行写操作时，就会触发页面中断，从而便可以在缺页中断处理函数中复制新页，达到写时复制的效果。

而页表的构造如下：

31	12	11	9	8	7	6	5	4	3	2	1	0
物理页的物理地址 31~12	AVL	G	PAT	D	A	PCD	PWT	US	RW	P		

RW 位即读写位，RW 为 1 时可读可写，RW 为 0 时只读。
因此在上面的 copyProcess 函数中，以下语句就是为页表设置只读位：

```
1 pageTableVaddr[j]&=0xffffffff; //设置为只读
2 int pte = pageTableVaddr[j];
3 int paddr=pte&0xfffff000; //设置页表项只读
```

Listing 38: 设置页表项只读位

那么，如何判断一个缺页中断是写只读页引起的中断呢？
这里利用到 X86 的缺页中断错误码：当页面中断发生时，x86 架构会把一个错误码压入栈中，描述了这次缺页访问的类型和原因，其中部分关键位号的含义如下：

位号	名称	含义
0	P(Present)	0= 页不在内存中 (缺页)，1= 页保护错误 (例如写只读页)
1	W/R	0= 读引发异常，1= 写访问引发异常
2	U/S	0= 内核态访问引发异常，1= 用户态访问引发异常

因此我们可以通过错误码获取到中断的触发方式，从而进行相应的中断处理。
先修改 asm_page_fault_handler 函数，使其在调用 c 中断处理函数时传入错误码作为参数

Listing 39: asm_page_fault_handler

```
1 asm_page_fault_handler:
2     ; 错误码已经在栈顶了
3     pushad
4     push dword [esp + 32] ; 将错误码作为参数传递 (pushad压入了8个寄存器=32字节)
5     call c_page_interrupt_handler
6     add esp, 4           ; 清理参数
7     popad
8     add esp, 4           ; 清理错误码
9     iret
```

因此在中断处理函数中，优先通过错误码判断是否是写访问只读页的错误，如果是的话就转到 COW 的处理：

```
1 extern "C" void c_page_interrupt_handler(uint32 error_code) //传入错误码
2 {
3     uint32 cr2=read_cr2();
```

```

4     cr2=cr2&0xfffff000;
5     // 分析错误码
6     bool page_present = error_code & 0x1;      // 位0: 页是否存在    为0则是缺页错误    为1则是页保护
        错误
7     bool write_access = error_code & 0x2;      // 位1: 是否为写访问
8     if (page_present && write_access) {
9         printf("COW page default on virtual address :0x%x\n",cr2);
10        memoryManager.COW_page_default_management(cr2);
11        return;
12    }

```

Listing 40: 中断处理函数：判断中断类型

由于有些处理还没考虑，COW_page_default_management 的实现下一节再展示。

只读页的引入增加了一种情况：当试图读访问只读页，但只读页在外存时，也会发生缺页错误（此时页表的存在位为 0，只读位为 0,31-12 位为物理页在外存的位置 pagenum）。

根据先前的处理逻辑，此时会分配物理页 → 建立页表联系 → 从外存调入数据

而建立页表联系对应的函数即为 connectPhysicalVirtualPage，在此函数中有以下语句

```
*pde = page | 0x7;
```

会导致页表的只读位改为 1，此页面可写，不符合逻辑。

在缺页处理函数中做出的修正如下：

```

1 int *pte=(int *)memoryManager.toPTE(cr2);
2 int *pde=(int *)memoryManager.toPDE(cr2);
3 int pagenum=0;
4 if (!(*pde & 0x00000001)&&!page_present) pagenum=0;    //无对应页表
5     else pagenum=(*pte)>>12;    //有对应页表
6 bool flag=true;
7 flag=memoryManager.connectPhysicalVirtualPage(cr2, physicalPageAddress);
8 if (pagenum!=0) {
9     bool read_only=((*pte)&0x2)>0?false:true;
10    if (read_only) {    //还要判断是否换入只读页
11        *pte&=0xfffffff;    //设置为只读
12    }
13    memoryManager.swapAreaManager.swapIn(cr2,pagenum);
14    printf("swap in page from disk\n");
15 }

```

Listing 41: 缺页处理函数：判断调入的页是否只读

新增一个 read_only 来判断。

6.5 物理页引用次数的维护

根据 6.2 的问题 3 和问题 4，在 COW 的处理中，我们必须要考虑原本的物理页的存在情况。当其余进程都进行了 COW，只剩一个进程时，这个物理页就可写了。因此新增一个数据结构，维护此物理页的被引用次数（即被多少个进程共享）

```

1 struct read_only_page {
2     int paddr;
3     int count;    //被引用次数
4     read_only_page(int paddr,int count):paddr(paddr),count(count){}

```

```

5     read_only_page(){}
6 };
7 struct Read_only_page_queue {
8     read_only_page queue[MAX_QUEUE_SIZE];
9     int front;
10    int rear;
11    Read_only_page_queue()
12    {
13        front=0;
14        rear=0;
15    }
16    int find_page(int paddr) {
17        for (int i=front;i!=rear;i=(i+1)%MAX_QUEUE_SIZE)
18        {
19            if (queue[i].paddr==paddr) {
20                return i;
21            }
22        }
23        return -1;
24    }
25    void add_page(int paddr) {
26        int index=find_page(paddr);
27        if (index==-1) {
28            queue[rear]=read_only_page(paddr,1);
29            rear=(rear+1)%MAX_QUEUE_SIZE;
30        } else {
31            queue[index].count++;
32        }
33    }
34    void pop_page(int paddr) {
35        int index=find_page(paddr);
36        if (index!=-1) {
37            queue[index].count--;
38        }
39    }
40 };

```

Listing 42: 只读页的引用维护

需要考虑的情况如下:

- 当一个进程写访问只读页时, 若此页的引用次数为 1, 则不分配新页, 将此页设为可写即可
- 当一个进程写访问只读页时, 若此页的引用次数大于 1, 则将此页的引用次数减一
- 当一个进程退出时, 若它使用的物理页的引用次数大于 1, 则不能释放这个物理页, 而是将物理页的引用次数减一

首先, 在上面的 copyProcess 函数中, 维护了共享物理页的引用计数:

```

1 for (int i=33;i<768;i++)//从 0x08400000 开始, 父子共享
2 {
3     // 无对应页表
4     if (!(parentPageDir[i] & 0x1))
5     {
6         continue;

```

```

7     }
8
9     // 计算页表的虚拟地址
10    int *pageTableVaddr = (int *) (0xffc00000 + (i << 12));
11
12    for (int j=0; j<1024; j++)
13    {
14        if (!(pageTableVaddr[j] & 0x1))
15        {
16            continue;
17        }
18        pageTableVaddr[j]&=0xffffffff; //设置为只读
19        int pte = pageTableVaddr[j];
20        int paddr=pte&0xfffff000;
21        int index=memoryManager.read_only_page_queue.find_page(paddr); //判断此页是否为共享页
22        if (index!=-1) memoryManager.read_only_page_queue.add_page(paddr); //若此页不是共享页,
23        //则要增加一次引用计数(父进程)
24        asm_update_cr3(childPageDirPaddr);
25        memoryManager.read_only_page_queue.add_page(paddr); //子进程增加一次引用计数
26        pageTableVaddr[j]=pte;
27        asm_update_cr3(parentPageDirPaddr);
28    }
29 }

```

Listing 43: copyProcess: 维护引用计数

而在进程的退出函数中，要增加关于引用页的维护

```

1 void ProgramManager::exit(int ret)
2 {
3     interruptManager.disableInterrupt();
4
5     PCB *program = this->running;
6     printf("exit, pid: %d\n", program->pid);
7     program->retValue = ret;
8     program->status = ProgramStatus::DEAD;
9
10    int *pageDir, *page;
11    int paddr;
12
13    if (program->pageDirectoryAddress)
14    {
15        pageDir = (int *)program->pageDirectoryAddress;
16        for (int i = 0; i < 768; ++i)
17        {
18            if (!(pageDir[i] & 0x1))
19            {
20                continue;
21            }
22
23            page = (int *) (0xffc00000 + (i << 12));
24
25            for (int j = 0; j < 1024; ++j)
26            {
27                if (!(page[j] & 0x1))

```

```

28         {
29             continue;
30         }
31
32         paddr = memoryManager.vaddr2paddr((i << 22) + (j << 12)); //对于物理页的处理
33         int index=memoryManager.read_only_page_queue.find_page(paddr);//要判断是不是被多个
34             进程引用的页
35         if (index!=-1)
36         {
37             memoryManager.read_only_page_queue.pop_page(paddr); //计数减一
38         }
39
40         if (index==-1||memoryManager.read_only_page_queue.queue[index].count==0) //若不是
41             共享页，或者只有一个进程共享，就释放
42             memoryManager.releasePhysicalPages(AddressPoolType::USER, paddr, 1);
43     }
44
45     paddr = memoryManager.vaddr2paddr((int)page);
46     memoryManager.releasePhysicalPages(AddressPoolType::USER, paddr, 1);
47 }
48 memoryManager.releasePages(AddressPoolType::KERNEL, (int)pageDir, 1);
49 int bitmapBytes = ceil(program->userVirtual.resources.length, 8);
50 int bitmapPages = ceil(bitmapBytes, PAGE_SIZE);
51 memoryManager.releasePages(AddressPoolType::KERNEL, (int)program->userVirtual.resources.
52     bitmap, bitmapPages);
53 }
54
55 schedule();
56 }

```

Listing 44: exit

6.6 COW 机制下的页面中断处理

因此，COW 的中断处理函数便可实现

```

1 void MemoryManager::COW_page_default_management(int vaddr)
2 {
3     int *pte=(int *)toPTE(vaddr);
4     int paddr=*pte&0xffff000;
5     int index=read_only_page_queue.find_page(paddr);
6     if (read_only_page_queue.queue[index].count<=1) { //若引用页小于1，则把此页变为可写，继续用此
7         页
8         *pte|=0x2;
9         return;
10    } else { //计数减一
11        read_only_page_queue.queue[index].count--;
12    }
13    char *buffer = (char *)malloc(PAGE_SIZE); //从内核分配一个交换页，存原页数据
14    memcpy((char *)vaddr,buffer,PAGE_SIZE);
15    //printf("buffer:%s\n",buffer);
16    int physicalPageAddress;
17    physicalPageAddress=allocatePhysicalPages(AddressPoolType::USER,1);
18    if (physicalPageAddress == 0) {

```



```

18     physicalPageAddress=exchangePage(vaddr);
19 }
20 UserphysicalPageQueue.push(item(vaddr,physicalPageAddress));
21 *pte|=0x2; //设置为可写
22 bool flag=connectPhysicalVirtualPage(vaddr,physicalPageAddress);
23 if (!flag) {
24     printf("connect with physical page failed\n");
25     asm_halt();
26 }
27 flush_tlb_single(vaddr);
28 memcpy(buffer,(char *)vaddr,PAGE_SIZE); //存入原页数据, 否则原页数据丢失
29 free(buffer,PAGE_SIZE);
30 }

```

Listing 45: COW_page_default_management

综上, 完整的缺页中断处理函数如下, 涉及缺页处理、帧满与外存交换处理、COW 处理

```

1 extern "C" void c_page_interrupt_handler(uint32 error_code)
2 {
3     uint32 cr2=read_cr2();
4     cr2=cr2&0xffff000;
5     // 分析错误码
6     bool page_present = error_code & 0x1; // 位0: 页是否存在 为0则是缺页错误 为1则是页保护
7     bool write_access = error_code & 0x2; // 位1: 是否为写访问
8     if (page_present && write_access) {
9         printf("COW page default on virtual address :0x%x\n",cr2);
10        memoryManager.COW_page_default_management(cr2);
11        return;
12    }
13    //printf("page fault on virtual address :0x%x\n",cr2);
14    if (!is_valid(cr2)) {
15        printf("Illegal memory access at 0x%x, process terminated\n", cr2);
16        asm_halt();
17        return;
18    }
19    // 合法地址, 为其分配物理页
20    int physicalPageAddress;
21    if (cr2 >= 0xC0000000) {
22        physicalPageAddress = memoryManager.allocatePhysicalPages(AddressPoolType::KERNEL, 1);
23    } else {
24        physicalPageAddress = memoryManager.allocatePhysicalPages(AddressPoolType::USER, 1);
25    }
26    if (physicalPageAddress == 0) {
27        physicalPageAddress=memoryManager.exchangePage(cr2);
28    }
29    // if (cr2>=KERNEL_VIRTUAL_START) {
30    //     memoryManager.KernelphysicalPageQueue.push(item(cr2,physicalPageAddress));
31    // }
32    // else
33    if (cr2>=USER_VADDR_START&&cr2<0xc0000000) {
34        memoryManager.UserphysicalPageQueue.push(item(cr2,physicalPageAddress));
35    }
36    int *pte=(int *)memoryManager.toPTE(cr2);

```

```

37     int *pde=(int *)memoryManager.toPDE(cr2);
38     int pagenum=0;
39     if (!(*pde & 0x00000001)&&!page_present) pagenum=0;    //无对应页表
40     else pagenum=(*pte)>>12;    //有对应页表
41     bool flag=true;
42     flag=memoryManager.connectPhysicalVirtualPage(cr2, physicalPageAddress);
43     if (pagenum!=0) {
44         bool read_only=((*pte)&0x2)>0?false:true;
45         if (read_only) {    //还要判断是否换入只读页
46             *pte&=0xffffffd;    //设置为只读
47         }
48         memoryManager.swapAreaManager.swapIn(cr2,pagenum);
49         printf("swap in page from disk\n");
50     }
51     if (!flag) {
52         printf("connect with physical page failed\n");
53         asm_halt();
54     }
55 }
56 }

```

Listing 46: 缺页中断处理

其中，由于原来代码过于冗长，页交换的函数单独封装了

```

1 uint32 MemoryManager::exchangepage(int vaddr)
2 {
3     uint32 physicalPageAddress;
4     // if (vaddr>=KERNEL_VIRTUAL_START) {    //内核态
5     //     if (KernelphysicalPageQueue.empty()) {
6     //         asm_halt();
7     //     }
8     //     item u=KernelphysicalPageQueue.getFront();
9     //     KernelphysicalPageQueue.pop();
10    //     swapOut(u.virtualAddress);
11    //     releasePhysicalPages(AddressPoolType::KERNEL, u.physicalAddress, 1);
12    //     physicalPageAddress=allocatePhysicalPages(AddressPoolType::KERNEL, 1);
13    //     flush_tlb_single(u.virtualAddress);
14    // }
15    // else
16    if (vaddr>=USER_VADDR_START&&vaddr<0xc0000000) {    //用户态
17        if (UserphysicalPageQueue.empty()) {
18            asm_halt();
19        }
20        item u=UserphysicalPageQueue.getFront();
21        UserphysicalPageQueue.pop();
22        swapOut(u.virtualAddress);
23        releasePhysicalPages(AddressPoolType::USER, u.physicalAddress, 1);
24        physicalPageAddress=allocatePhysicalPages(AddressPoolType::USER, 1);
25        flush_tlb_single(u.virtualAddress);
26    }
27    return physicalPageAddress;
28 }
29 }

```

Listing 47: 页交换函数

6.7 测试样例

由于之前为了方便，把共享地址部分设成了第 33 个页目录开始，那么开始共享的虚拟地址是 0x8400000。为了便于测试，先在进程中空 malloc 一段地址，使得接下来 malloc 返回的地址处于父子进程的共享区中：

```
1 malloc(0x3b7*4096);
2 //从 0x08400000 开始，父子共享
```

Listing 48: 空 malloc

6.7.1 测试样例一

本样例测试写时复制的实现

```
1 void second_process()
2 {
3     malloc(0x3b7*4096);
4     //从 0x08400000 开始，父子共享
5     int retval;
6     char *p=(char *)malloc(15);
7     strcpy("hello",p);
8     int pid=fork();
9     if (pid==0) {
10         uint32 paddr=getpaddr((int)p);
11         printf("child before COW: vaddr:%x paddr:%x %s\n",p,paddr,p);
12         //父进程已经写时复制，子进程不复制
13         strcpy("sysu",p);
14         paddr=getpaddr((int)p);
15         printf("child after COW: vaddr:%x paddr:%x %s\n",p,paddr,p);
16         //再写，这次不复制
17         strcpy("good afternoon",p);
18         paddr=getpaddr((int)p);
19         printf("child after COW2: vaddr:%x paddr:%x %s\n",p,paddr,p);
20         free(p,15);
21     } else {
22         uint32 paddr=getpaddr((int)p);
23         printf("parent before COW: vaddr:%x paddr:%x %s\n",p,paddr,p);
24         //写时复制
25         strcpy(" world",p+5);
26         paddr=getpaddr((int)p);
27         printf("parent after COW: vaddr:%x paddr:%x %s\n",p,paddr,p);
28         //再写，这次不复制
29         strcpy("good morning",p);
30         paddr=getpaddr((int)p);
31         printf("parent after COW2: vaddr:%x paddr:%x %s\n",p,paddr,p);
32         printf("\n\n\n");
33         free(p,15);
34     }
35     asm_halt();
36 }
```

Listing 49: 样例一: second_process

首先在 fork 之前创建共享的指针 p，内容为“hello”，父进程在 hello 后面写入“world”（测试 COW 是否正确复制共享页的内容），再写入“good morning”；子进程写入“sysu”，再写入“good afternoon”

输出如下

```

Machine  View
parent before COW: vaddr:8400000 paddr:4420000 hello
COW page default on virtual address :0x8400000
parent after COW: vaddr:8400000 paddr:47E4000 hello world
parent after COW2: vaddr:8400000 paddr:47E4000 good morning

child before COW: vaddr:8400000 paddr:4420000 hello
COW page default on virtual address :0x8400000
child after COW: vaddr:8400000 paddr:4420000 sysu
child after COW2: vaddr:8400000 paddr:4420000 good afternoon

```

分析此输出：

1. 在写入之前，对两个进程分别访问了 p 指向的空间经过转化后的物理地址，可以看到父子进程的物理页是共享的。
2. 父进程先执行，写操作时发生了 COW 中断，复制物理页到新的地址，可以看到物理地址发生了变化，随后访问该地址内容，成功得到 hello world。原先页的内容被成功复制，新加入的内容成功写入。
3. 父进程再写，物理地址没改变，说明新的页是可写的，成功写入。
4. 接着到子进程执行，子进程成功写入，发生 COW 中断，但物理地址没改变，因为父进程已经进行了 COW，此物理页的引用次数为 1，子进程不创建新页，而是继续用这个页。
5. 子进程再写，没有 COW 中断，成功写入。

6.7.2 测试样例二

这次测试一个父进程，多个子进程的情况：

```

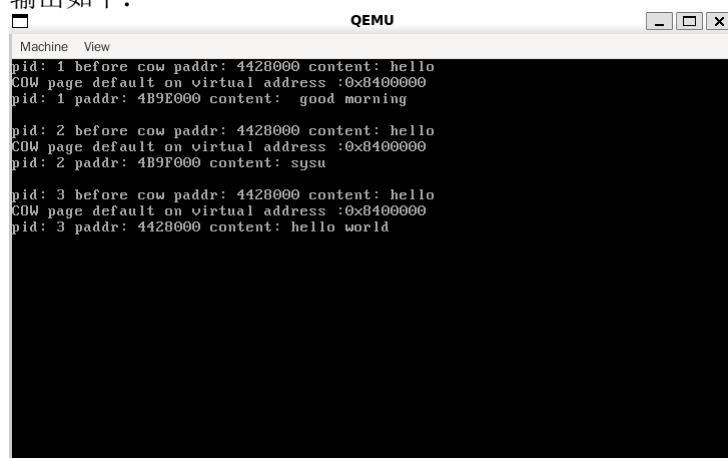
1 int getcurpid()
2 {
3     return programManager.running->pid;
4 }
5 void third_process()
6 {
7     malloc(0x3b7*4096);
8     int retval;
9     char *p=(char *)malloc(15);
10    strcpy("hello",p);
11    int pid=fork();
12    if (pid==0) {
13        printf("pid: %d before cow paddr: %x content: %s\n",getcurpid(),getpaddr((int)p),p);
14        strcpy("sysu",p);
15        printf("pid: %d paddr: %x content: %s\n",getcurpid(),getpaddr((int)p),p);
16        printf("\n");
17    } else {
18        int pid2=fork();
19        if (pid2==0) {
20            printf("pid: %d before cow paddr: %x content: %s\n",getcurpid(),getpaddr((int)p),p);

```

```
21     strcpy(" world",p+5);
22     printf("pid: %d paddr: %x content: %s\n",getcurpid(),getpaddr((int)p),p);
23     printf("\n");
24 } else {
25     // wait(&retval);
26     // wait(&retval);
27     printf("pid: %d before cow paddr: %x content: %s\n",getcurpid(),getpaddr((int)p),p);
28     strcpy(" good morning",p);
29     printf("pid: %d paddr: %x content: %s\n",getcurpid(),getpaddr((int)p),p);
30     printf("\n");
31 }
32 }
33 asm_halt();
34 }
```

Listing 50: 样例二：一个父进程，多个子进程

输出如下：



```
Machine View
pid: 1 before cow paddr: 4428000 content: hello
COW page default on virtual address :0x8400000
pid: 1 paddr: 4B9E000 content: good morning

pid: 2 before cow paddr: 4428000 content: hello
COW page default on virtual address :0x8400000
pid: 2 paddr: 4B9F000 content: sysu

pid: 3 before cow paddr: 4428000 content: hello
COW page default on virtual address :0x8400000
pid: 3 paddr: 4428000 content: hello world
```

可以看到，三个进程最初都是共享一片物理页，前两个写入的进程都创建了新页，最后一个写入的页直接利用此物理页，表明页引用计数的设置没有问题。

7 实验总结与心得体会

经过一个学期的操作系统原理与实验的学习，Project5 为本学期操作系统实验的最终成品。它以相对较少的代码实现了一个简单的操作系统内核，但“麻雀虽小五脏俱全”，涵盖了大部分的操作系统基础知识点。既通过代码的编写实现巩固了本学科的原理性知识，也锻炼了工程项目的能力，让我受益匪浅。

在最终成果的 Project5 中，集齐了以下功能：

- **Lab3：由实模式到保护模式的跳转：**本实验的程序在保护模式环境下运行。
- **Lab3：以 LBA 方式读取磁盘：**将磁盘 IO 封装在了 disk.h 中，实现了与外存的交互，在 Project1 的内存管理完善中发挥了至关重要的作用。
- **Lab4：保护模式的中断：**实现了时钟中断和页面错误中断的注册、以及编写中断处理程序，利用中断进一步实现了内存管理的请求调页、写时复制、时间片轮转的线程调度等算法。
- **Lab5：内核线程：**我们的调度是基于线程的调度，用户进程的创建过程是在内核线程的基础之上，用户进程需要通过关联内核线程实现与操作系统的联系。
- **Lab6：锁和信号量：**在页面分配、malloc 等函数中，进程/线程是必须互斥进行的，这需要用到自旋锁、信号量等方法，否则会因为同步等问题发生不可预见的错误。

图 1: OS 实验项目代码量

```
● (base) david@David:~/i386/Lab9/project5$ cloc .
    41 text files.
    39 unique files.
    25 files ignored.

github.com/AlDanial/cloc v 1.98  T=0.05 s (861.8 files/s, 94336.2 lines/s)
-----
Language                      files      blank      comment      code
-----
C++                            13         355         205         2068
C/C++ Header                   20         135         125         619
Assembly                       4          133         41          499
make                            1           17           1           50
Markdown                       1           4            0           17
-----
SUM:                           39         644         372         3253
-----
```

- **Lab7: 内存管理:** 本项目实现了二级分页内存管理，同时还在 Lab7 的基础上扩展了请求调页、页面置换到外存、以及在写时复制中物理页引用次数的维护等功能，进一步完善了该 OS 内核。
- **Lab8: 从内核态到用户态:** 本项目对用户进程和内核态进行了隔离，并通过系统调用的方式使得用户进程可以进行 printf、malloc、free、fork、wait、exit 等功能。并进一步在这些功能的基础上扩展了写时复制。
- **Lab9: 虚拟内存完善、malloc 和 free 的实现、写时复制:** 这三个功能是我对示例的扩展，并最终集成到了 Project5 中。

在实验中，利用 gdb 进行调试是一项非常重要的 debug 手段，特别是 fork、以及写时复制的实现过程中，出现了许许多多奇怪的错误，需要通过 gdb 进行一步步的 debug 找出错误。

同时，通过 OS 实验，我还掌握了 makefile 的使用，掌握了对大型项目的管理，这些知识让我受益匪浅。

最后，要感谢实验文档的编写者以及指导老师陈鹏飞老师。文档编写的非常细致，对实现 OS 内核项目的所有技术栈 (qemu、gdb、x86 汇编、makefile 等基本编程知识)，和每个功能的每一处实现细节都描述的非常到位，能够让我们在零疑惑中轻松掌握一个简易 OS 内核的编写。